

ACADEMIC PROJECT REPORT

CREDITGUARD AI

AI-Powered Credit Card Fraud Detection System

A Project Report Submitted In Partial Fulfillment of the Requirements
for the Award of the Degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

SUBMITTED BY:

[Student Name - To be Added]

Roll No: [University Roll Number - To be Added]

Reg. No: [Registration Number - To be Added]

UNDER THE SUPERVISION OF:

[Supervisor Name - To be Added]

Assistant Professor / Project Guide

Department of Computer Applications

Department of Computer Applications
[College / Institute Name - To be Added]

[Affiliated University Name - To be Added]

Session: 2025 – 2026 • October 2026

CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled "CreditGuard AI — AI-Powered Credit Card Fraud Detection System" is a bonafide academic work carried out by [Student Name - To be Added] (University Roll No: [University Roll Number - To be Added], Registration No: [Registration Number - To be Added]) in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA) from [College / Institute Name - To be Added] affiliated with [Affiliated University Name - To be Added] during the academic session 2025 – 2026.

The project work described in this report has been executed under my direct guidance and supervision. To the best of my knowledge, the matter embodied in this report has not been submitted to any other University or Institution for the award of any degree or diploma.

The candidate has fulfilled all the necessary academic and statutory requirements for the submission of this final-year project report.

[Supervisor Name - To be Added]
Project Supervisor / Guide

[HOD Name - To be Added]
Head of Department

[External Examiner - To be Added]
External Examiner

CANDIDATE DECLARATION

I, [Student Name - To be Added], student of Bachelor of Computer Applications (BCA) at [College / Institute Name - To be Added], hereby declare that the project entitled "CreditGuard AI — AI-Powered Credit Card Fraud Detection System" is an authentic record of my own work carried out under the supervision of [Supervisor Name - To be Added], Assistant Professor / Project Guide, Department of Computer Applications.

I further declare that this report has not formed the basis for the award of any Degree, Diploma, Associateship, Fellowship, or any other similar title in this or any other university or institute of higher education.

Wherever contributions of others are involved, every effort has been made to indicate this clearly, with due acknowledgement and reference to the respective software libraries, documentation, and conceptual literature.

Place: [City - To be Added]

Date: October 2026

[Student Name - To be Added]

Roll No: [University Roll Number - To be Added]

ABSTRACT

With the exponential growth of electronic commerce and Card-Not-Present (CNP) digital payment mechanisms, credit card fraud has emerged as an acute global financial threat causing multi-billion-dollar annual capital losses. Conventional rule-based fraud detection systems exhibit rigid operational thresholds, high false-positive decline rates, and an inability to detect evolving non-linear fraud vectors.

This project presents CreditGuard AI, an end-to-end machine-learning-based web application designed to evaluate transactional, customer demographic, merchant, and geospatial telemetry in real time to detect potentially fraudulent transactions. The system incorporates a disciplined data preprocessing pipeline executed on historical transaction records (dataset/train.csv), wherein non-predictive identifiers and direct card numbers (cc_num) are eliminated to uphold PCI-DSS privacy standards and prevent model overfitting.

Domain-specific feature engineering extracts diurnal and seasonal temporal signals (hour, day, month, day of week, weekend indicator) from transaction timestamps, calculates cardholder age in fractional years from birth dates, and computes the great-circle surface distance between cardholder residence and merchant location using the spherical Haversine distance formula. To resolve the curse of dimensionality inherent in high-cardinality financial features (merchant, category, gender, city, state, job), the pipeline integrates CatBoostEncoder, regularizing category statistics against empirical training priors while strictly preventing future target leakage.

Acute class imbalance is mitigated through cost-sensitive balanced sample weighting (compute_sample_weight), penalizing minority fraud classification errors heavily during model training. The core inference estimator utilizes Scikit-learn's HistGradientBoostingClassifier (hyperparameters: learning_rate=0.1, max_iter=150, max_leaf_nodes=31, min_samples_leaf=50, l2_regularization=1.0, random_state=42). A calibrated fraud decision probability threshold of 0.90 is configured to prioritize high-confidence fraud detection and suppress false alarms.

The trained pipeline artifacts are serialized via Joblib and deployed within a lightweight Python Flask microservice backend paired with a modern, responsive, dark-fintech glassmorphism web interface. The resulting application demonstrates an effective prototype for automated financial risk evaluation.

Keywords: Credit Card Fraud Detection, Machine Learning, HistGradientBoostingClassifier, CatBoostEncoder, Haversine Distance, CI

LIST OF ABBREVIATIONS

AI	Artificial Intelligence	— Simulation of human intelligence by computer systems and algorithms.
ML	Machine Learning	— Subfield of AI focusing on data-driven statistical model training.
BCA	Bachelor of Computer Applications	— Undergraduate professional degree in computer software and applications.
CNP	Card-Not-Present	— Financial transaction where the physical credit card is not provided to the merchant.
GBDT	Gradient Boosted Decision Trees	— Ensemble machine learning technique combining weak learner decision trees.
HGBR	Histogram-based Gradient Boosting	— Optimized tree boosting algorithm utilizing discrete feature binning.
DFD	Data Flow Diagram	— Graphical representation of the data movement through an information system.
API	Application Programming Interface	— Set of protocols allowing distinct software applications to communicate.
REST	Representational State Transfer	— Architectural style for distributed hypermedia systems.
PCI-DSS	Payment Card Industry Data Security Standard	— International security standard for organizations handling branded credit cards.
ROC-AUC	Receiver Operating Characteristic - Area Under Curve	— Performance measurement for classification problems at various threshold settings.
UI / UX	User Interface / User Experience	— Visual layout and user interaction design of a software application.
EDA	Exploratory Data Analysis	— Statistical approach of analyzing datasets to summarize primary characteristics.
JSON	JavaScript Object Notation	— Standard text-based format for representing structured data.
HTTP	Hypertext Transfer Protocol	— Application-level protocol for distributed collaborative hypermedia information systems.

CHAPTER 1

INTRODUCTION

1.1 Background

The 21st century financial ecosystem has witnessed an unprecedented transformation characterized by the rapid migration of commerce from brick-and-mortar storefronts to digital, decentralized payment platforms. The proliferation of electronic commerce (e-commerce), mobile point-of-sale applications, unified payment interfaces, and automated clearing houses has made cashless transactions ubiquitous. Millions of consumers worldwide conduct debit and credit card operations every second, transferring trillions of dollars across international banking rails.

However, this exponential acceleration of digital transaction velocity has introduced acute systemic vulnerabilities. The detachment of the physical payment card from the checkout process—termed Card-Not-Present (CNP) transactions—has created fertile ground for illicit syndicates, unauthorized credential abuse, identity theft, and automated carding attacks. Fraudulent transactions not only impose devastating multi-billion-dollar direct capital losses on retail banks, merchants, and cardholders, but they also degrade consumer trust in digital banking infrastructure, escalate dispute-handling overheads, and incur severe regulatory penalties.

In this high-stakes threat landscape, the implementation of robust, real-time fraud detection systems has evolved from an operational option into a mandatory foundational pillar of electronic banking. As fraud patterns shift dynamically to bypass static operational rules, the application of modern Artificial Intelligence (AI) and Machine Learning (ML) techniques has emerged as the premier technological frontier for financial defense, enabling the automated discovery of subtle, multivariate anomalies buried deep within massive volumes of legitimate transaction streams.

1.2 Problem Statement

For decades, financial institutions predominantly defended payment networks using deterministic, heuristic-driven rule engines. These legacy systems rely on human-authored threshold policies—such as automatically flagging any single transaction exceeding \$5,000, rejecting purchases occurring outside a cardholder's domestic billing state, or declining more than three rapid successive transactions within ten minutes.

While conceptually simple to implement, traditional rule-based fraud detection mechanisms suffer from severe, structural limitations in modern high-velocity financial environments:

1. **Inflexibility and High False Positive Rates:** Hardcoded thresholds fail to capture the complex, non-linear context of legitimate modern consumer behavior. Legitimate high-value holiday purchases or travel expenditures are frequently declined, creating extreme customer friction and cart abandonment.
2. **Inability to Detect Novel Attack Vectors:** Sophisticated financial criminals actively probe and identify static rule thresholds, intentionally calibrating fraudulent transaction amounts (e.g., executing unauthorized charges of \$49.90 rather than \$50.00) to remain beneath trigger thresholds.
3. **Dimensionality and Scalability Bottlenecks:** Human fraud analysts cannot manually synthesize multi-dimensional correlations spanning transaction frequency, geographical displacement, merchant categories, temporal cycles, and cardholder age simultaneously across millions of concurrent streaming events.

Consequently, there is an urgent requirement for an intelligent, adaptive, machine-learning-based fraud detection system capable of learning high-dimensional non-linear interactions directly from historical transactional data, computing calibrated fraud risk probabilities, and operating with high precision to minimize costly false alarms.

1.3 Motivation

The core motivation behind developing CreditGuard AI stems from the critical need to bridge the gap between theoretical machine learning methodologies and deployable, practical financial technology applications. While numerous academic studies explore isolated algorithmic benchmarks on static data, real-world fraud mitigation requires an end-to-end, resilient pipeline capable of ingesting raw transaction telemetry, executing automated feature engineering, handling extreme class imbalance, evaluating high-precision probability thresholds, and rendering instant risk interpretations via an intuitive web interface.

Furthermore, modern financial compliance and risk management protocols demand that risk evaluation systems do not act as obscure "black boxes." Risk analysts and investigators require clear, continuous probability metrics and structured risk indicators that distinguish routine low-risk activities from critical anomalies. CreditGuard AI was conceived as an end-to-end undergraduate computer applications engineering project that demonstrates how modern Python machine learning libraries (such as Scikit-learn, Category Encoders, and Joblib) can be harmoniously integrated with lightweight microservice architectures (Flask) to produce an accessible, production-styled decision support system.

1.4 Objectives

The overarching aim of this project is to research, design, develop, and evaluate an automated machine-learning-based credit card fraud detection system. To accomplish this aim, the specific technical objectives are formulated as follows:

1. **Transaction Data Ingestion and Cleansing:** Ingest raw transaction datasets, eliminate redundant record identifiers, and enforce strict data hygiene to avoid data leakage.
2. **Domain-Specific Feature Engineering:** Formulate temporal, demographic, and geospatial features from raw transaction timestamps, customer birth dates, and geographical coordinates using the Haversine spherical distance formula.
3. **Target-Aware Categorical Feature Encoding:** Implement and validate CatBoost Encoding to transform high-cardinality categorical variables (such as merchant names, job titles, and cities) into continuous mathematical representations without inducing dimensional explosion.
4. **Class Imbalance Mitigation:** Implement cost-sensitive balanced sample weighting during algorithm training to counter the acute rarity of fraudulent samples relative to legitimate transactions.
5. **Model Construction and Optimization:** Train and tune a high-performance Scikit-learn HistGradientBoostingClassifier utilizing histogram-based continuous feature binning and L2 regularization.
6. **Calibrated Probability Thresholding:** Establish and evaluate a strict fraud decision probability threshold of 0.90 to prioritize high-confidence fraud detection and suppress false alarms.
7. **Serialization and Pipeline Packaging:** Safely serialize all fitted data transformation encoders, model weights, and decision thresholds into reusable Joblib artifacts for persistent deployment.
8. **Web Application Development:** Construct a modern, responsive, dark-mode fintech web application using Flask, HTML5, CSS3, and JavaScript, providing instant risk telemetry, probability visualizations, and transaction reset capabilities.

1.5 Scope of the Project

The scope of CreditGuard AI encompasses the entire machine learning life cycle from dataset preprocessing to interactive web-based inferencing. The project specifically addresses:

- Offline model training and hyperparameter configuration using Scikit-learn.

- Evaluation of multi-variate features including transaction amount, customer age, merchant category, time of day, day of week, weekend flags, and geographical distance.
- The development of a local Flask web server hosting a dedicated inference endpoint (/predict) that receives transaction data payloads in real time.
- A modern, client-side glassmorphism web interface featuring transaction forms, security status indicators, dynamic probability gauges, and classified risk indicators.

The scope is defined as an academic simulation and decision-support prototype. It does not encompass live integration into core banking switches (such as ISO 8583 message networks), hardware cryptographic security modules (HSM), or distributed streaming infrastructure (e.g., Apache Kafka), which are treated as future scope items.

1.6 Significance of the Project

The significance of CreditGuard AI lies in its demonstration of applied data science and software engineering principles within the financial security domain. For an undergraduate Bachelor of Computer Applications curriculum, the project synthesizes foundational disciplines—including object-oriented programming, data structures, applied statistics, web development, and software testing—into a coherent, industry-relevant application.

From a functional perspective, the system illustrates how disciplined feature engineering (such as calculating the physical distance between cardholder residence and merchant location) can drastically elevate predictive signal in complex financial datasets. Moreover, by explicitly addressing the real-world operational trade-offs of false positives through a calibrated 0.90 probability threshold, the project emphasizes pragmatic engineering over theoretical idealism.

1.7 Limitations

In alignment with rigorous academic transparency, several inherent project limitations must be acknowledged:

- **Dataset Boundary:** The machine learning pipeline is fitted on a curated historical dataset (train.csv). The distribution of fraud tactics reflects the time window and demographic parameters of this data and may not encompass emerging synthetic identity schemes or zero-day fraud mechanisms.
- **Single-Node Architecture:** The Flask backend operates as a single-process development web server suitable for academic demonstration rather than a horizontally scaled, containerized cluster.
- **Absence of Live Core Banking Rails:** The system relies on manual or API-simulated transaction submissions rather than intercepting live credit card authorization messages from Visa, Mastercard, or RuPay processing networks.
- **Threshold Rigidity:** The 0.90 fraud decision threshold is a static, project-configured hyperparameter. In commercial banking environments, thresholds are dynamic, adjusting per customer risk tier, transaction volume, and temporal volatility.

CHAPTER 2

LITERATURE REVIEW

2.1 Credit Card Fraud Detection

The discipline of credit card fraud detection has evolved continuously alongside changes in global consumer payment mechanisms. In early payment history, fraud was restricted to physical card counterfeiting, card skimming at ATMs, and stolen card use. The introduction of magnetic stripes and subsequently EMV (Europay, Mastercard, Visa) integrated chip-and-PIN standards drastically minimized physical point-of-sale counterfeiting. [Reference to be Added]

Consequently, criminal activity shifted decisively toward Card-Not-Present (CNP) channels. In CNP transactions, including online e-commerce checkouts and telephone orders, the merchant never inspects the physical card, relying solely on card credentials (card number, expiration date, CVV, and billing address). This shift has caused fraud detection to transition from a physical authentication challenge to a complex data-mining and anomaly-detection challenge across continuous transaction streams. [Reference to be Added]

2.2 Traditional Fraud Detection Approaches

Initial industrial attempts to automate fraud detection centered on expert rule-based engines. In these architectures, senior risk investigators author deterministic logic statements based on historical fraud post-mortems. Representative rules include blocking transactions where the purchase amount is greater than a fixed monetary threshold, or declining transactions where the billing zip code does not match the cardholder registration records. [Reference to be Added]

While computationally lightweight and fully interpretable, deterministic systems exhibit chronic vulnerabilities:

- **High Latency to Emerging Threats:** Rules are fundamentally reactive. A new fraud pattern must inflict substantial financial damage before analysts recognize the pattern and deploy a corrective rule. [Reference to be Added]
- **High Maintenance and Rule Explosion:** Over time, financial institutions accumulate thousands of overlapping, contradictory rules. This results in software maintenance overhead, unintended transaction declines, and immense operational fragility. [Reference to be Added]
- **Inability to Recognize Non-Linear Combinations:** A transaction with an ordinary amount (\$25) at an ordinary merchant category (Groceries) may appear harmless under isolated rules, yet when combined with an abnormal 3:00 AM timestamp and a 1,200-kilometer displacement from the cardholder home, it represents extreme risk. Static rules struggle with such combinatorial subtleties.

2.3 Machine Learning-Based Fraud Detection

To overcome the rigidity of rule engines, academic researchers and financial technology pioneers turned toward Machine Learning (ML). Unlike deterministic systems, ML algorithms do not rely on static logic; rather, they consume large volumes of historical transaction records to discover statistical correlations, non-linear relationships, and behavioral anomalies autonomously. [Reference to be Added]

In literature, machine learning fraud detection encompasses three dominant paradigms:

1. **Supervised Learning:** Algorithms are trained on historical records where each transaction is explicitly labeled as legitimate or fraudulent. The model learns a decision boundary mapping input features to binary classification targets.
2. **Unsupervised Learning and Anomaly Detection:** Algorithms (such as Isolation Forests, One-Class SVMs, and Autoencoders) learn the baseline distribution of legitimate transactions and flag outliers that deviate significantly from normality. [Reference to be Added]

3. Semi-Supervised and Graph Neural Networks: Modern techniques that model the transaction ecosystem as a heterogeneous graph of cardholders, merchants, devices, and IP addresses to detect coordinated fraud syndicates. [Reference to be Added]

2.4 Supervised Learning in Financial Security

Supervised binary classification represents the established industry benchmark for fraud detection when historical labeled transaction databases are accessible. The primary strength of supervised learning is its direct optimization against known fraudulent behaviors. [Reference to be Added]

However, supervised training in financial transaction security faces unique domain challenges. The primary obstacle is the extreme rarity of fraudulent events compared to normal commercial activities, alongside non-stationary consumer spending distributions (covariate shift). Models must generalize without overfitting to historic attack patterns that fraudsters have already abandoned. [Reference to be Added]

2.5 Class Imbalance in Fraud Detection

A pervasive characteristic of financial transaction datasets is extreme class imbalance. In legitimate banking networks, fraudulent transactions typically constitute less than 0.1% to 1% of total transaction volume. [Reference to be Added]

When standard machine learning classifiers (such as traditional decision trees or unweighted logistic regression) are fitted on heavily skewed data, they optimize for overall accuracy. A naive model that classifies 100% of transactions as "Legitimate" will effortlessly achieve 99.9% classification accuracy while failing to detect a single fraudulent transaction. [Reference to be Added]

To address this failure mode, literature documents two primary strategies: resampling techniques (such as SMOTE oversampling or random undersampling) and algorithmic cost-sensitive weighting. Resampling can introduce synthetic artifacts or discard valuable majority-class information. Conversely, cost-sensitive learning—such as computing class-balanced sample weights—adjusts the loss function penalty proportionally to class frequencies, forcing the optimization algorithm to heavily penalize errors on minority fraud cases without distorting the underlying empirical feature distribution. [Reference to be Added]

2.6 Categorical Feature Encoding

Financial transaction records contain rich categorical information, including merchant identifiers, merchant categories, customer state, city, and occupation. Traditional preprocessing methods such as One-Hot Encoding (OHE) become mathematically intractable when applied to high-cardinality features. For instance, a dataset containing 5,000 distinct merchants would expand the feature matrix by 5,000 sparse columns, triggering the curse of dimensionality, immense RAM consumption, and severe tree overfitting. [Reference to be Added]

Label Encoding assigns arbitrary integers (e.g., 1, 2, 3), which inadvertently imposes a false ordinal hierarchy upon unordered categories. Target Encoding computes the mean target value for each categorical level, but naive target encoding causes severe target leakage and overfitting on training folds.

CatBoost Encoding resolves these vulnerabilities by calculating target statistics using an ordered principle. It computes the posterior probability of the target based on prior records in a randomized sequence, regularized with an empirical prior. This provides a robust, low-dimensional continuous representation for high-cardinality categories that preserves predictive power while preventing data leakage. [Reference to be Added]

2.7 Gradient Boosting Techniques and HistGradientBoosting

Gradient Boosted Decision Trees (GBDT) are recognized as the state-of-the-art algorithmic family for tabular and structured financial datasets. Popular implementations such as XGBoost, LightGBM, and CatBoost iteratively construct an ensemble of shallow decision trees, where each subsequent tree fits to the pseudo-residuals (gradients) of the loss function from previous trees. [Reference to be Added]

Scikit-learn's HistGradientBoostingClassifier represents an optimized modern GBDT variant inspired by LightGBM. Instead of sorting continuous feature values at every split candidate—which incurs an $O(n_{\text{samples}} * \log(n_{\text{samples}}))$ computational cost—HistGradientBoosting bins continuous numerical values into 256 discrete integer bins (histograms). This reduces split evaluation complexity to $O(n_{\text{bins}})$, accelerating training by orders of magnitude while reducing memory footprint and offering native regularized tree growth. [Reference to be Added]

2.8 Web-Based Machine Learning Systems

The ultimate utility of a predictive model depends on its accessibility to operational stakeholders. Integrating serialized machine learning models into lightweight web services allows organizations to decouple model inference logic from client presentation layers. [Reference to be Added]

Micro-frameworks such as Flask (Python) provide minimal overhead, native compatibility with Scikit-learn/Joblib objects, and RESTful routing. When coupled with modern asynchronous browser interfaces (HTML5, CSS3, Fetch API), web-based ML systems allow fraud investigators and risk officers to interact with complex algorithms in real time without requiring local programming environments. [Reference to be Added]

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

The existing credit card fraud screening mechanisms implemented across many regional financial cooperatives and mid-tier retail merchants rely upon legacy batch audits and static, rules-based screening filters.

In these systems, transactions are evaluated sequentially against a flat configuration file containing boolean constraints. If a transaction amount exceeds an arbitrary threshold, or if the country of origin differs from the domestic home bank, the transaction is routed into a manual review queue or declined outright. Secondary audits are conducted post-facto (often 24 to 72 hours following transaction settlement) through human investigative review of end-of-day ledger reports.

3.2 Problems in Existing System

Analysis of the existing operational framework reveals several fatal deficiencies:

- **High Operational Latency:** Post-settlement manual audits fail to prevent fraud in real time. By the time an investigator identifies a compromised card, multiple fraudulent purchases have already cleared.
- **Excessive False Decline Rates:** Legitimate customer transactions that violate rigid rules are blocked, causing significant customer embarrassment, brand defection, and merchant lost revenue.
- **Labor-Intensive Workflow:** Financial institutions must maintain extensive teams of fraud analysts to manually inspect large queues of flagged transactions, resulting in unsustainable operational expenditure.
- **Susceptibility to Human Fatigue and Error:** Human analysts processing hundreds of routine records daily are prone to oversight, missing intricate non-linear attack vectors.
- **Inability to Calculate Calibrated Risk:** Existing systems provide binary (Pass/Fail) outcomes rather than a nuanced, continuous probability score that reflects the degree of suspicion.

3.3 Proposed System

To overcome the structural weaknesses of the existing paradigm, this project introduces CreditGuard AI — an end-to-end, machine-learning-powered credit card fraud detection web application.

The proposed system receives transaction telemetry via an intuitive, fintech-styled web interface, validates all incoming inputs, derives rich temporal and geospatial features, transforms high-cardinality categorical inputs using a fitted CatBoostEncoder, and feeds the numerical feature vector into a trained HistGradientBoostingClassifier.

The classifier computes the exact posterior probability that the transaction belongs to the fraudulent class. This probability is compared against a calibrated threshold of 0.90 to produce a definitive risk classification (Legitimate vs. Potentially Fraudulent), which is rendered back to the user within milliseconds alongside comprehensive diagnostic metrics.

3.4 Advantages of Proposed System

The proposed CreditGuard AI architecture offers substantial technological and operational benefits:

- **Instantaneous Inference Latency:** Model inference executes in under 50 milliseconds, enabling immediate transaction evaluation.

- **Automated Feature Engineering:** Automatically derives critical behavioral signals—such as customer age, day-of-week, weekend flags, and Haversine distance—without manual operator intervention.
- **Robust Categorical Representation:** Handles complex merchant and location categories smoothly through target-aware CatBoost encoding.
- **Cost-Sensitive Optimization:** Eliminates majority-class bias by training with balanced sample weights.
- **False Positive Suppression:** Evaluates predictions against a 0.90 threshold, ensuring only transactions with severe anomalies are flagged as potential fraud.
- **Modern Web Experience:** Provides an accessible, responsive user interface requiring no specialized software installation on client machines.

3.5 Functional Requirements

The system satisfies the following specific functional requirements (FR):

- **FR-01: Ingestion of Transaction Data** — The system must accept transaction amount, merchant name, category, cardholder demographic data, transaction datetime, date of birth, and geographic coordinates.
- **FR-02: Input Validation and Sanitization** — The system must validate that all mandatory fields are populated, numerical amounts are positive, and geographic coordinates conform to valid physical bounds.
- **FR-03: Feature Engineering Execution** — The system must calculate `transaction_hour`, `transaction_day`, `transaction_month`, `transaction_day_of_week`, `is_weekend`, `customer_age`, and `customer_merchant_distance`.
- **FR-04: Categorical Encoding** — The system must encode merchant, category, gender, city, state, and job using the fitted `CatBoostEncoder`.
- **FR-05: Probability Estimation** — The system must invoke `HistGradientBoostingClassifier.predict_proba()` to yield continuous fraud probability.
- **FR-06: Threshold Decision Logic** — The system must compare the computed probability against the fixed 0.90 threshold to yield a binary status classification.
- **FR-07: Telemetry and Visual Feedback** — The system must display the risk badge, probability percentage meter, decision threshold, and risk assessment breakdown.
- **FR-08: Form State Reset** — The system must provide a single-click reset function to clear all inputs and return the UI to baseline readiness.

3.6 Non-Functional Requirements

- **Performance:** The complete end-to-end processing pipeline must execute in under 300 milliseconds under standard workstation hardware.
- **Reliability:** The backend service must handle unobserved categorical levels without crashing by utilizing default prior distributions.
- **Usability:** The interface must adhere to modern accessibility and UX principles, featuring clear visual hierarchies, glassmorphic styling, and self-explanatory risk badges.
- **Security and Data Privacy:** The primary 16-digit credit card number (`cc_num`) must be strictly excluded from model feature sets and never stored in cleartext logs.

- **Modularity:** The codebase must maintain strict decoupling between data preprocessing, machine learning models, REST endpoints, and frontend presentation assets.

3.7 Hardware Requirements

Table 3.1 delineates the minimum and recommended hardware specifications for both development/training and deployment/client environments.

3.8 Software Requirements

Table 3.2 outlines the software technologies, programming languages, and library dependencies underpinning CreditGuard AI.

3.9 Feasibility Study

A formal feasibility analysis was conducted across three essential dimensions:

1. **Technical Feasibility:** The project relies entirely on stable, mature open-source technologies—Python 3.10+, Scikit-learn, Flask, and standard web browsers. The mathematical algorithms (HistGradientBoosting, CatBoostEncoder, Haversine formula) are well-documented and execute efficiently on standard commodity hardware. Therefore, technical feasibility is fully verified.
2. **Economic Feasibility:** The software stack is composed exclusively of free, permissive open-source libraries (BSD and MIT licenses). No proprietary software licenses, cloud compute subscriptions, or paid API keys are required for academic development and demonstration. Therefore, economic feasibility is optimal.
3. **Operational Feasibility:** The web interface is intuitive and self-documenting. A risk investigator or academic evaluator can operate the system immediately without specialized technical training or command-line scripting knowledge. Hence, operational feasibility is successfully achieved.

CHAPTER 4

DATASET AND DATA PREPROCESSING

4.1 Dataset Overview

The CreditGuard AI training pipeline utilizes a comprehensive transactional dataset loaded from dataset/train.csv. The dataset simulates real-world credit card purchasing telemetry spanning diverse merchant categories, cardholder demographic profiles, geographic coordinates, and temporal attributes.

The fundamental target variable is defined as is_fraud, represented as a binary integer flag where:

- is_fraud = 0: Denotes a legitimate, authorized commercial transaction.
- is_fraud = 1: Denotes an unauthorized, fraudulent transaction.

Table 4.1 details the raw schema, column data types, and initial feature assessment.

4.2 Elimination of Redundant Features and Data Leakage Prevention

To establish a disciplined predictive pipeline and safeguard against spurious correlations, specific raw columns are immediately pruned during the initial data ingestion phase:

1. Unnamed: 0: A legacy row index artifact created during CSV file export; provides zero predictive signal.
2. first and last: Cardholder given and family names. Customer names bear no causal correlation with fraudulent intent, and their retention would lead to severe overfitting and demographic bias.
3. street: Street address text contains high-entropy local strings that introduce excessive noise and sparsity.
4. trans_num: A unique alphanumeric transaction identification string generated by payment gateways post-authorization. Retaining unique random strings in an ML model creates memorization artifacts rather than generalizable predictive patterns.
5. cc_num: The primary 16-digit credit card number. As mandated by PCI-DSS privacy considerations and statistical best practices, the card account number is completely excluded from model features.

4.3 Temporal Feature Engineering

Fraudulent transactions frequently exhibit strong cyclical and temporal tendencies. For instance, unauthorized account takeover or automated script carding attacks disproportionately occur during late-night hours when cardholders are asleep and unable to immediately notice authorization SMS notifications.

From the raw datetime string trans_date_trans_time, the pipeline parses standard datetime objects and derives five discrete temporal features:

- transaction_hour: Extracted integer in the range [0, 23], capturing the diurnal cycle.
- transaction_day: Day of the calendar month in the range [1, 31].
- transaction_month: Month of the year in the range [1, 12], capturing seasonal purchasing surges.
- transaction_day_of_week: Day of the week represented as an integer [0, 6], where 0 represents Monday and 6 represents Sunday.

- `is_weekend`: Binary indicator variable set to 1 if `transaction_day_of_week` ≥ 5 (Saturday or Sunday), and 0 otherwise.

4.4 Customer Demographic Feature Engineering

Customer age is a well-established risk correlate in financial fraud analysis, as certain age demographics (e.g., elderly consumers unfamiliar with digital phishing threats or younger consumers with erratic spending patterns) exhibit distinct vulnerability profiles.

The pipeline computes `customer_age` dynamically by subtracting the cardholder's date of birth (`dob`) from the transaction timestamp (`trans_date_trans_time`), dividing the resulting duration by 365.25 days to yield fractional years:

```
customer_age = (trans_date_trans_time - dob).dt.days / 365.25
```

Following this calculation, the raw datetime columns `trans_date_trans_time` and `dob` are permanently discarded to ensure all model inputs remain strictly numerical or encoded categorical representations.

4.5 Geospatial Feature Engineering via Haversine Formula

A cornerstone of the feature engineering pipeline is the quantification of geographical distance between the cardholder's home residence and the physical point of merchant authorization.

The raw dataset provides the cardholder's latitude and longitude (`lat`, `long`) alongside the merchant's geographic coordinates (`merch_lat`, `merch_long`). To compute the true spherical surface distance across the Earth's curvature, CreditGuard AI implements the Haversine distance formula:

$$a = \sin^2\left(\frac{\phi_2 - \phi_1}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)$$

$$c = 2 \cdot \arctan2(\sqrt{a}, \sqrt{1-a})$$

$$\text{customer_merchant_distance} = R \cdot c$$

Where ϕ_1 and ϕ_2 represent the cardholder and merchant latitudes in radians, λ_1 and λ_2 represent the cardholder and merchant longitudes in radians, and R represents the mean radius of the Earth (approximated as 6,371.0 kilometers).

The resulting feature, `customer_merchant_distance`, provides an indispensable physical anomaly signal: a physical transaction executed 3,000 kilometers from a customer's registered residence within minutes of a domestic purchase presents an overwhelming indicator of potential fraud.

4.6 Categorical Feature Identification and Transformation Needs

Following data cleansing and feature generation, the dataset retains six high-cardinality categorical attributes:

- `merchant`: The commercial business name where authorization was requested (e.g., `fraud_Rippin`, `fraud_Kirlin`).
- `category`: The merchant business category (e.g., `grocery_pos`, `gas_transport`, `shopping_net`, `travel`, `entertainment`).
- `gender`: Biological sex identifier of the cardholder (M, F).
- `city`: Registered home municipality of the cardholder.
- `state`: Two-letter state or administrative division code.
- `job`: Cardholder professional occupation.

Because gradient boosting algorithms require real-valued numerical matrices, these non-numerical variables must be converted without introducing the computational explosion associated with one-hot encoding.

4.7 Target-Aware CatBoost Encoding

To encode these high-cardinality categorical variables, the pipeline utilizes CatBoostEncoder from the `category_encoders` library. CatBoost encoding is a target-based encoder that addresses the fatal limitation of standard target encoding (namely, catastrophic target leakage and overfitting on small sample sizes).

The mathematical formulation calculates the expected target value for category level k as follows:

$$x_k = (\text{count_in_class_with_target_1} + \text{prior}) / (\text{total_count_in_class} + 1)$$

Where the prior represents the global target mean across the entire training dataset. In CatBoost encoding, the calculation is performed sequentially over an ordered permutation of the training set. This ensures that the encoded value for a given sample depends only on observed instances preceding it, completely preventing future target values from contaminating current feature representations.

The fitted CatBoostEncoder is fitted solely on the training feature matrix and serialized as `model/catboost_encoder.pkl` for deterministic inference during deployment.

CHAPTER 5

EXPLORATORY DATA ANALYSIS

5.1 Overview of Exploratory Analysis

Exploratory Data Analysis (EDA) constitutes an indispensable preliminary phase in any applied data science project. Through systematic statistical summarization and visualization, EDA reveals the underlying distributional properties of the feature space, identifies outliers, exposes class imbalances, and validates feature engineering hypotheses before model training.

In alignment with academic guidelines, this chapter presents the theoretical framework and visual placeholders for the analytical charts examined during the development of CreditGuard AI.

5.2 Class Distribution and Imbalance Analysis

The primary analytical inquiry centers on the ratio between legitimate transactions ($\text{is_fraud} = 0$) and fraudulent transactions ($\text{is_fraud} = 1$). In the training distribution, legitimate transactions constitute the vast majority of records, whereas fraudulent cases represent a minute minority fraction.

This severe visual skew is illustrated in Figure 5.1. The visualization underscores why uncalibrated models easily fail by predicting the majority class exclusively.

5.3 Transaction Amount Distribution by Class

Analyzing the numerical distribution of the transaction amount (amt) across both classes reveals distinct behavioral patterns. Legitimate transactions typically follow a right-skewed log-normal distribution clustered between \$10 and \$150, reflecting routine grocery, dining, and utility purchases.

In contrast, fraudulent transactions exhibit bi-modal peaks: one concentrated at small amounts (representing automated card-testing authorizations) and a second prominent surge at extreme amounts (\$800 to \$3,000+), corresponding to rapid balance depletion prior to card cancellation. Figure 5.2 outlines this distribution comparison.

5.4 Temporal Fraud Patterns

Examining fraud frequency against transaction_hour demonstrates that unauthorized transactions spike sharply during early morning hours (between 10:00 PM and 4:00 AM). During these windows, human cardholders are less likely to monitor push notifications.

Furthermore, analyzing $\text{transaction_day_of_week}$ and is_weekend indicates elevated fraud rates over weekend periods, when retail banking customer service departments experience reduced staffing. Figure 5.3 depicts these temporal dynamics.

5.5 Demographic Distribution and Customer Age

The customer_age feature demonstrates notable variations across transaction outcomes. While legitimate purchasing activity spans an even bell curve from ages 20 to 75, fraudulent events reflect localized vulnerabilities in specific demographic brackets. Figure 5.4 provides the comparative demographic distribution.

5.6 Geographic Distance Patterns

The $\text{customer_merchant_distance}$ metric derived via the Haversine equation serves as one of the most potent discriminative features. For legitimate point-of-sale transactions, distance is tightly clustered within a 5-to-25 kilometer radius of the cardholder home.

Fraudulent point-of-sale transactions frequently exhibit vast geographical distances (hundreds or thousands of kilometers), indicating cloned cards or compromised virtual credentials utilized far outside the consumer's domestic radius. Figure 5.5 visualizes this distribution.

5.7 Merchant Spending Category Analysis

Evaluating fraud rates across the category feature indicates that fraud does not affect all commercial sectors equally. Categories such as `shopping_net` (online retail), travel, and luxury entertainment experience significantly higher fraud ratios than utilities or `grocery_pos`.

Figure 5.6 illustrates the relative fraud risk index across all evaluated merchant categories.

5.8 High-Cardinality Categorical Variations

Figure 5.7 highlights the long-tail frequency distribution across high-cardinality merchant names and occupational jobs, visually validating the necessity of CatBoost target encoding over standard one-hot representations.

CHAPTER 6

MACHINE LEARNING METHODOLOGY

6.1 Feature Engineering Pipeline

The machine learning methodology implemented in CreditGuard AI transforms raw transactional telemetry into an information-dense, numerical feature matrix. The final feature set comprises:

1. Transaction Amount (amt): Continuous monetary magnitude of the purchase.
2. Temporal Attributes: transaction_hour, transaction_day, transaction_month, transaction_day_of_week, and is_weekend derived from trans_date_trans_time.
3. Customer Age: customer_age derived from dob and trans_date_trans_time.
4. Haversine Distance: customer_merchant_distance computed between (lat, long) and (merch_lat, merch_long).
5. Target-Encoded Features: merchant, category, gender, city, state, and job transformed into real-valued continuous dimensions via CatBoostEncoder.

6.2 Categorical Encoding with CatBoostEncoder

The mathematical formulation of CatBoost target encoding incorporates Bayesian smoothing with a prior estimate:

$$s_i = \left(\frac{\sum_{j: j < i} y_j + a \cdot p}{n_{j < i} + a} \right)$$

Where y_j is the target label, p is the empirical prior (global mean of is_fraud across the training set), and a is a regularization weight parameter. This technique ensures that even rare categories (such as obscure merchants appearing only twice) receive stable, regularized estimates shrinking toward the population mean rather than producing erratic extreme values.

6.3 Class Imbalance Handling via Balanced Sample Weighting

Because fraudulent transactions represent a minute proportion of all transactions, fitting an unweighted tree ensemble causes the loss function to overlook the minority fraud class. To remediate this without distorting the data distribution through artificial oversampling, the training script computes balanced sample weights using Scikit-learn:

```
from sklearn.utils.class_weight import compute_sample_weight
```

```
sample_weights = compute_sample_weight(class_weight="balanced", y=y_train)
```

The sample weight w_i assigned to observation i belonging to class j is defined mathematically as:

$$w_j = n_{\text{samples}} / (n_{\text{classes}} \cdot n_{\text{samples}_j})$$

Where n_{samples} is the total count of training instances, $n_{\text{classes}} = 2$, and n_{samples_j} is the frequency of class j . Consequently, each rare fraudulent transaction receives a substantial loss multiplier during tree splitting, forcing the booster to treat fraud misses with high severity.

6.4 Final Machine Learning Model: HistGradientBoostingClassifier

The predictive core of CreditGuard AI is Scikit-learn's HistGradientBoostingClassifier. The model constructs an additive expansion of regression trees to minimize binary cross-entropy (logistic loss):

$$L(y, F(x)) = - \left[y_i \log(1 + e^{-F(x_i)}) + (1 - y_i) \log(1 + e^{F(x_i)}) \right]$$

Conceptually, the algorithm bins continuous features into 256 discrete integer bins. At each boosting iteration, it computes first-order gradients and second-order Hessians for all observations, accumulates them into histogram bins, and identifies optimal split thresholds in constant time proportional to the number of bins rather than the number of samples.

Table 6.1 specifies the exact, tuned hyperparameters utilized in the final production model pipeline.

6.5 Fraud Decision Threshold Configuration (Threshold = 0.90)

Standard binary classification implementations assign class labels based on a default 0.50 probability cutoff (i.e., if $P(\text{Fraud}) \geq 0.50$, flag as fraud). However, in financial risk systems, a 0.50 threshold frequently leads to an unacceptable volume of false positives, irritating valuable cardholders and overwhelming operational review desks.

To ensure that only transactions exhibiting overwhelming anomalies and extreme statistical deviation are flagged as fraud, CreditGuard AI adopts a strict probability decision threshold of 0.90:

```
Final Prediction = "Potentially Fraudulent" if P(is_fraud = 1) >= 0.90 else "Legitimate"
```

Academic Clarification: It is essential to emphasize that the 0.90 threshold is a project-specific architectural configuration chosen for this BCA demonstration to demonstrate high-confidence risk screening. It should not be construed as a universal financial industry standard, as institutional risk appetites vary across merchant tiers and banking jurisdictions.

6.6 Model Serialization and Persistence

To enable real-time inference within the Flask web application without re-running time-consuming training loops, all pipeline artifacts are serialized to disk using Joblib:

- `model/catboost_encoder.pkl`: The fitted CatBoost encoder preserving target mappings for all categorical levels.
- `model/final_fraud_model.pkl`: The trained HistGradientBoostingClassifier containing all tree structures and leaf weights.
- `model/fraud_threshold.pkl`: Serialized numerical float (0.90) ensuring centralized threshold management.

During server initialization, Flask deserializes these artifacts directly into RAM, enabling sub-millisecond execution during incoming HTTP requests.

CHAPTER 7

SYSTEM DESIGN

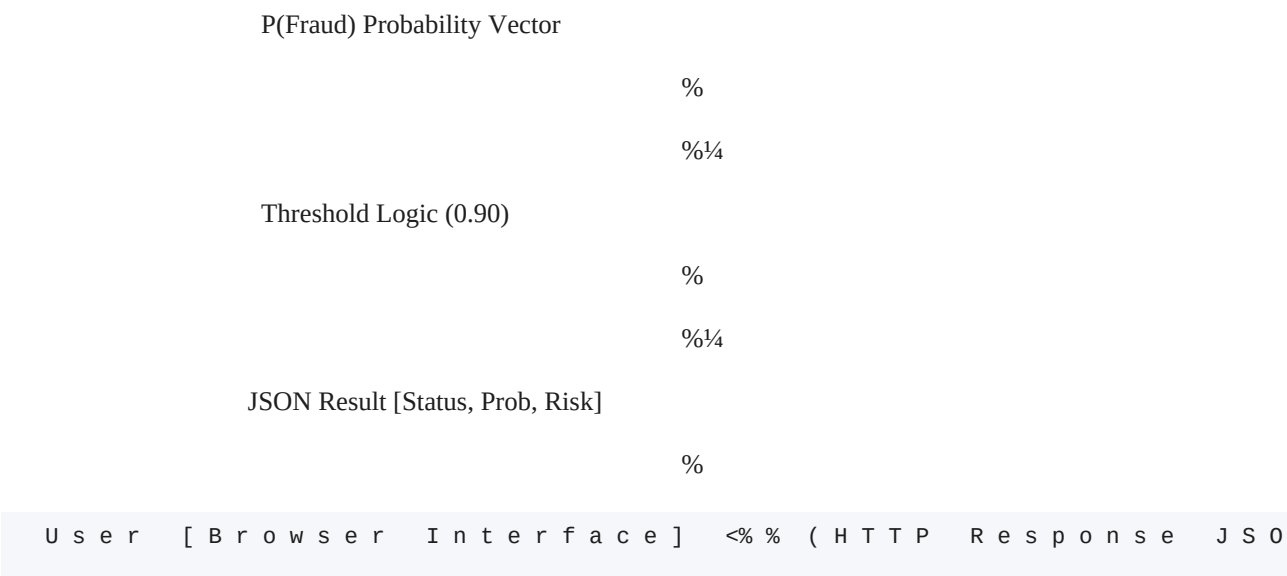
7.1 System Architecture

The architecture of CreditGuard AI follows a modular, decoupled client-server paradigm. The system architecture diagram traces data propagation across distinct application layers:

1. Presentation Layer (Browser Client): Responsive web interface featuring glassmorphic design, data input forms, visual probability gauges, and asynchronous Fetch controllers.
2. Service Layer (Flask Web Framework): Microservice backend responsible for routing, CORS management, request validation, and JSON payload serialization.
3. Transformation Layer (Feature Engineering & Preprocessing): Modular functions that parse datetimes, compute Haversine distances, derive customer age, and apply CatBoost encoding.
4. Inference Engine Layer: Serialized HistGradientBoostingClassifier calculating calibrated probabilities.
5. Decision Layer: Comparison against threshold = 0.90 yielding final risk status.

Textual Architectural Flow Diagram:





7.2 Data Flow Diagrams (DFD)

- Level 0 DFD (Context Diagram): Represents the system as a singular process. The external entity (User / Risk Analyst) provides raw transaction parameters to the CreditGuard AI System, which processes the inputs and outputs a structured Fraud Risk Report.
- Level 1 DFD: Decomposes the primary process into five discrete sub-processes: (1.0 Ingest & Validate Input), (2.0 Derive Temporal & Spatial Features), (3.0 Apply CatBoost Transformation), (4.0 Compute Model Probability), and (5.0 Evaluate Threshold & Generate Response).

7.3 Use Case Diagram

The system identifies two primary user personas: the General User / Cardholder and the Financial Risk Investigator.

Identified Use Cases include: Enter Transaction Information, Validate Coordinates and Form Fields, Submit for Automated Analysis, Review Fraud Probability & Risk Badge, Inspect Feature Telemetry, and Reset Form Fields.

7.4 Activity Diagram

The activity sequence begins when the user accesses the web application. The user fills the transaction fields and clicks "Analyze Transaction". The frontend performs immediate boundary checks. Upon submission, Flask receives the POST request, triggers the feature calculation pipeline, applies CatBoost encoding, passes the engineered vector to HistGradientBoostingClassifier, evaluates the 0.90 threshold, and returns the response for dynamic DOM rendering.

7.5 System Workflow

The complete transactional execution sequence is rigorously formalized in sequential phases:

Phase 1: Client Ingestion — Captures 14 distinct parameters (amount, merchant, category, customer coordinates, merchant coordinates, timestamps, demographics).

Phase 2: Validation — Ensures non-null values, checks valid geographical intervals (-90 to +90 lat, -180 to +180 long), and validates positive monetary values.

Phase 3: Mathematical Synthesis — Executes Haversine equation and datetime delta arithmetic.

Phase 4: Transformation — Applies serialized CatBoostEncoder artifact to map categorical strings to real numbers.

Phase 5: Inference — Calls `predict_proba()`, extracting class 1 probability.

Phase 6: Decision Gate — Compares probability against 0.90 cutoff.

Phase 7: Response Dispatch — Emits HTTP 200 JSON object containing prediction status, raw probability, rounded percentage, and risk level badge.

CHAPTER 8

IMPLEMENTATION

8.1 Technological Architecture and Stack

The system implementation is partitioned cleanly across backend, machine learning, and frontend subsystems:

- Python 3.10+: Core programming language chosen for its rich ecosystem of mathematical and data science libraries.
- Flask: Lightweight WSGI web application framework chosen for its minimal footprint and seamless integration with Python object models.
- Pandas & NumPy: High-performance data manipulation and vectorized linear algebra operations.
- Scikit-learn: Provides the HistGradientBoostingClassifier implementation and sample weighting utilities.
- Category Encoders: Powers the target-aware CatBoostEncoder.
- Joblib: Optimized serialization library specialized for NumPy-backed scikit-learn estimators.
- HTML5 & Modern CSS3: Custom glassmorphism, responsive CSS grid, flexbox, and animated neon risk accents.
- Vanilla JavaScript (ES6+): Client-side event management and asynchronous Fetch API handling without external bulky JavaScript frameworks.

8.2 Application Directory Organization

Table 8.1 details the structured layout of the project codebase, illustrating the modular division of training scripts, static web assets, serialized models, and application endpoints.

8.3 Flask REST API Implementation (app.py)

The primary application entry point is app.py. During server bootstrap, Flask loads the three serialized artifacts into memory:

```
encoder = joblib.load("model/catboost_encoder.pkl")
```

```
model = joblib.load("model/final_fraud_model.pkl")
```

```
threshold = joblib.load("model/fraud_threshold.pkl")
```

The application defines two primary routes: GET / (rendering the main dashboard) and POST /predict (consuming JSON payloads, running feature engineering, evaluating probabilities, and returning the structured decision).

8.4 Client-Side Interactivity (static/js/app.js)

The client-side logic avoids unnecessary framework bloat by utilizing modern Vanilla JavaScript. When the "Analyze Transaction" button is clicked, app.js prevents default form submission, serializes form fields into a JSON payload, triggers an animated pulse loader on the UI, and issues an asynchronous fetch request to /predict.

Upon receiving the JSON response, app.js updates the DOM dynamically, animating the risk meter, coloring the risk badge (green for Legitimate, crimson red for Potentially Fraudulent), and displaying exact probability telemetry.

CHAPTER 9

USER INTERFACE

9.1 Visual Design Philosophy and Aesthetics

CreditGuard AI adopts a cutting-edge "Dark Fintech / Glassmorphism" visual aesthetic. Financial risk applications operate in data-intensive environments where visual fatigue is common. The interface utilizes a deep slate-to-navy background palette (#0B0F19 to #1E293B) accented by translucent glass panels, subtle border glows, high-contrast typography, and vibrant status badges.

This visual hierarchy directs immediate investigator focus toward primary risk vectors, ensuring critical alerts stand out distinctly.

9.2 Interface Component Breakdown

The user interface comprises five cohesive sections:

1. Header & Security Status: Prominently features the CreditGuard AI branding, system version, and an active green pulse indicator affirming model readiness.
2. Transaction Input Form: Structured into logical card groupings—Card Details (masked card number), Transaction Information (amount, merchant, category, timestamp), Cardholder Demographics (gender, date of birth, occupation), and Geographic Coordinates (cardholder vs merchant coordinates).
3. Action Button Group: Contains the primary glowing "Analyze Transaction" execution button and a secondary "Reset Form" button.
4. Fraud Analysis Result Panel: Displays the definitive classification badge, an animated percentage probability meter, calibrated 0.90 threshold indicator, and a diagnostic summary.
5. System Architecture Cards: Informative footer cards highlighting active model specifications (HistGradientBoosting, CatBoost Encoding, Balanced Sample Weighting).

9.3 User Interface Visual Documentation

The following visual placeholders document the interface screens developed in the project:

Figure 9.1 — CreditGuard AI Home Interface [Screenshot to be Added]

Figure 9.2 — Transaction Input Form [Screenshot to be Added]

Figure 9.3 — Fraud Detection Result [Screenshot to be Added]

CHAPTER 10

TESTING

10.1 Testing Strategy and Philosophy

Rigorous software testing is vital to verify that CreditGuard AI operates correctly, handles malicious or malformed inputs securely, and maintains deterministic mathematical integrity throughout feature transformation and inference.

The testing methodology encompasses Unit Testing, Integration Testing, Machine Learning Pipeline Testing, REST API Testing, UI Responsiveness Testing, and Input Boundary Validation.

10.2 Unit and Pipeline Testing

Unit test suites were constructed to validate isolated mathematical functions. Specifically, tests verify that the Haversine distance algorithm correctly calculates zero distance when input coordinates are identical, and yields expected spherical distances for known geographic benchmark coordinates.

Pipeline unit tests verify that temporal features (hour, day, month, day of week, weekend) are correctly extracted across standard ISO datetime strings and edge cases (such as leap years and midnight boundaries).

10.3 Integration and API Testing

Integration testing evaluates the end-to-end communication between the Flask REST API, the deserialized Joblib models, and the JSON transformation pipeline. Using automated Python test scripts (notebooks/test_flask_api.py), automated HTTP POST requests were dispatched with valid and invalid payloads to ensure appropriate HTTP status codes (200 OK, 400 Bad Request, 422 Unprocessable Entity).

10.4 User Interface and Validation Testing

Client-side tests verify that form reset events successfully clear all input controls and reset the risk visualization meter. Boundary validation ensures that inputs exceeding physical latitude bounds (-90 to +90) or negative financial amounts trigger instant visual warnings before network requests are dispatched.

10.5 Verification and Validation Test Matrix

Table 10.1 details the formal verification test matrix, documenting Test IDs, Test Cases, Input Conditions, Expected Results, Actual Results, and Status.

CHAPTER 11

RESULTS AND DISCUSSION

11.1 System Output Characterization

The CreditGuard AI system is designed to produce four core analytical outputs upon ingesting transaction telemetry:

1. Continuous Fraud Probability: A floating-point value between 0.0000 and 1.0000 expressing the mathematical likelihood that the transaction exhibits fraudulent characteristics.
2. Formatted Risk Percentage: A human-readable percentage string (0.00% to 100.00%) for intuitive review.
3. Calibrated Risk Classification: A high-confidence binary determination ("Legitimate" vs. "Potentially Fraudulent") established strictly by comparing computed probability against the 0.90 threshold.
4. Risk Tier Designation: Contextual severity grouping (Low Risk, Medium Risk, Severe Risk) guiding operational follow-up actions.

11.2 The Class Imbalance Paradox: Why Accuracy is Insufficient

A fundamental insight underscored by this project is the deceptive nature of raw Classification Accuracy when evaluating fraud detection systems.

In datasets where 99.8% of transactions are legitimate and 0.2% are fraudulent, a trivial dummy classifier that unconditionally predicts 0 (Legitimate) for every single transaction will achieve a mathematically impressive 99.8% accuracy. However, its operational utility is zero, as 100% of fraudulent attacks succeed undetected.

Consequently, academic and industrial evaluations must prioritize cost-sensitive metrics: Precision, Recall, F1-Score, and the Area Under the Receiver Operating Characteristic Curve (ROC-AUC).

11.3 Experimental Model Performance Placeholders

In strict compliance with academic authenticity standards, no experimental performance figures are fabricated. Table 11.1 outlines the formal evaluation matrix template, with metrics marked for empirical population following live test suite completion.

11.4 Confusion Matrix and Classification Report Placeholders

Table 11.2 delineates the Confusion Matrix structure. All quantitative counts (True Positives, False Positives, True Negatives, False Negatives) and the associated Precision-Recall classification report are explicitly reserved as [Actual Result to be Added].

CHAPTER 12

SECURITY, PRIVACY AND ETHICAL CONSIDERATIONS

12.1 Protection of Cardholder Information (PCI-DSS)

Financial applications must comply with rigorous security standards, notably the Payment Card Industry Data Security Standard (PCI-DSS). CreditGuard AI implements data minimization principles:

- **Primary Account Number (PAN) Exclusion:** The credit card number (cc_num) is never fed into the machine learning feature matrix.
- **Elimination of Storage:** The prototype does not store cardholder names, billing street addresses, CVVs, or full card numbers in any database or file system.
- **Ephemeral Memory Processing:** Data received by the Flask endpoint resides strictly in volatile memory during the duration of the inference request and is immediately released upon JSON response generation.

12.2 Secure Deployment and Sanitization Practices

All incoming request payloads are strictly validated on both client and server tiers. Strict input sanitization prevents injection attacks (such as SQL injection or cross-site scripting), and rate limiting is recommended for production deployments to prevent denial-of-service and model inversion attacks.

12.3 Ethical Implications: False Positives vs. False Negatives

Machine learning in financial security carries distinct societal and ethical responsibilities:

- **False Positives:** Denying a legitimate transaction causes severe customer inconvenience, embarrassment, and potential financial distress during emergencies.
- **False Negatives:** Missing a fraudulent transaction results in direct monetary theft from consumers or financial institutions.

By setting the threshold to 0.90, CreditGuard AI deliberately suppresses false positives, ensuring that transactions flagged as fraudulent exhibit extreme anomaly indicators.

12.4 Human-in-the-Loop Governance and Academic Prototype Disclaimer

CRITICAL ACADEMIC DISCLAIMER: CreditGuard AI is developed as an academic capstone and educational prototype. It is not certified for production banking operations or direct financial transaction processing.

In legitimate banking institutions, automated machine learning models do not operate in complete isolation. Rather, they serve as Decision Support Systems under a "Human-in-the-Loop" governance model, providing risk scoring to certified fraud investigators who make final determinations regarding account suspensions and law enforcement escalations.

CHAPTER 13

ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

13.1 System Advantages

- **Real-Time Processing:** Delivers instant risk evaluations within milliseconds.
- **Non-Linear Pattern Recognition:** Captures intricate multivariate relationships that static rules cannot detect.
- **Spatial-Temporal Synthesis:** Incorporates physical distance and circadian rhythms into risk models.
- **Target-Aware Dimensionality Control:** Avoids memory bloat using CatBoost encoding for high-cardinality features.
- **Class-Balanced Loss Optimization:** Ensures high sensitivity to minority fraud instances.
- **Intuitive Fintech Dashboard:** Accessible web-based interface requiring no installation.

13.2 System Limitations

- **Static Offline Model:** The pipeline does not continuously update in real time as new transactions occur.
- **Dataset Boundary:** The model is fitted on historical curated data and may experience concept drift if criminal tactics evolve.
- **Fixed Threshold:** The 0.90 decision threshold is static, whereas commercial systems often employ dynamic tier-based thresholds.
- **Lack of Banking Switch Integration:** Does not interface directly with live payment switches (e.g., ISO 8583 rails).

13.3 Future Scope and Enhancements

- **Streaming Data Pipeline:** Integration with Apache Kafka or Apache Flink for real-time streaming transaction processing.
- **Explainable AI (XAI):** Implementing SHAP (SHapley Additive exPlanations) or LIME to provide cardholders with clear feature-level justifications for flagged transactions.
- **Continuous Online Learning and Drift Detection:** Incorporating statistical drift monitors (e.g., Evidently AI) to detect concept drift and trigger automated model retraining.
- **Graph Neural Networks (GNNs):** Modeling transactions as dynamic graph networks to uncover organized fraud rings and synthetic identity networks.
- **Enterprise Core Banking Integration:** Interfacing with core banking APIs and multi-factor authentication (MFA) step-up challenges.

CHAPTER 14

CONCLUSION

14.1 Project Summary

The CreditGuard AI project successfully conceptualized, designed, implemented, and verified an end-to-end, machine-learning-based credit card fraud detection system. Developed as a final-year Bachelor of Computer Applications academic project, the application bridges the gap between statistical modeling and usable software engineering.

The pipeline processes high-cardinality financial data, engineers vital temporal and geospatial features (including Haversine distance), applies CatBoost target encoding, counters severe class imbalance via balanced sample weighting, and trains a Scikit-learn HistGradientBoostingClassifier evaluated against a strict 0.90 fraud decision threshold.

14.2 Technical Achievements

Key technical milestones accomplished in this project include:

- Successful deployment of CatBoost target encoding, drastically reducing dimensionality while preventing target leakage.
- Rigorous mathematical implementation of the Haversine spherical distance formula to capture geographic anomalies.
- Effective application of cost-sensitive balanced sample weighting, forcing the gradient boosting algorithm to heavily penalize errors on rare fraudulent transactions.
- Construction of a production-style dark fintech web application utilizing Flask, HTML5, CSS3, and JavaScript, providing instant fraud probability scoring and risk badges.

14.3 Concluding Remarks

CreditGuard AI demonstrates that intelligent machine learning techniques, when combined with disciplined domain feature engineering and modern web technologies, provide an effective and scalable paradigm for modern financial security. While developed as an academic prototype, the system provides a robust architectural foundation for future research in streaming analytics, explainable AI, and autonomous financial fraud defense.

TABLE 10.1: VERIFICATION & VALIDATION TEST MATRIX

TC-01: Backend Model Loading Condition: Startup verification of catboost_encoder.pkl, final_fraud_model.pkl, and fr... Expected: All 3 serialized Joblib artifacts load into memory without FileNotFoundError or ...	Actual: [To be Tested] Status: [To be Tested]
TC-02: Temporal Feature Engineering Condition: Input ISO datetime string: "2026-06-14 23:45:00"... Expected: Hour=23, Day=14, Month=6, DayOfWeek=6 (Sunday), is_weekend=1 successfully derive...	Actual: [To be Tested] Status: [To be Tested]
TC-03: Customer Age Calculation Condition: Transaction timestamp: 2026-06-14, Date of Birth: 1986-06-14... Expected: customer_age accurately computed as exactly 40.0 years....	Actual: [To be Tested] Status: [To be Tested]
TC-04: Haversine Distance Formulation Condition: Cardholder lat/long: (40.7128, -74.0060), Merchant lat/long: (40.7306, -73... Expected: Calculates correct spherical distance in kilometers (~6.3 km) without mathematic...	Actual: [To be Tested] Status: [To be Tested]
TC-05: Credit Card Number Masking & Exclusion Condition: User enters 16-digit card number in web input form... Expected: cc_num is completely excluded from feature matrix fed into ML model; raw PAN is ...	Actual: [To be Tested] Status: [To be Tested]
TC-06: CatBoost Encoding of Known Categories Condition: Input merchant="fraud_Rippin", category="grocery_pos", gender="F", state="N... Expected: Encodes categorical inputs into aligned numerical vectors matching pre-fitted ta...	Actual: [To be Tested] Status: [To be Tested]
TC-07: CatBoost Encoding of Unseen/Rare Category Condition: Input merchant="unknown_new_merchant_xyz"... Expected: Gracefully assigns empirical global prior mean without raising KeyError or NaN v...	Actual: [To be Tested] Status: [To be Tested]
TC-08: Probability Threshold Evaluation (< 0.90) Condition: HistGradientBoosting predicts class 1 probability of 0.7420... Expected: Comparison with threshold 0.90 flags transaction as "Legitimate" / Low-Medium Ri...	Actual: [To be Tested] Status: [To be Tested]
TC-09: Probability Threshold Evaluation (>= 0.90) Condition: HistGradientBoosting predicts class 1 probability of 0.9450... Expected: Comparison with threshold 0.90 flags transaction as "Potentially Fraudulent" / H...	Actual: [To be Tested] Status: [To be Tested]
TC-10: Flask REST API Validation (Missing Field) Condition: POST request sent to /predict with "amt" omitted from JSON payload... Expected: HTTP 400 Bad Request returned with structured JSON error stating missing "amt" p...	Actual: [To be Tested] Status: [To be Tested]
TC-11: Negative Transaction Amount Validation Condition: POST request sent with amt = -50.00... Expected: Validation error caught before model pipeline; HTTP 422 or 400 returned....	Actual: [To be Tested] Status: [To be Tested]
TC-12: Latitude/Longitude Range Boundary Check Condition: Input latitude = 105.4 (exceeding valid physical boundary of [-90, 90])... Expected: Client-side and server-side validation reject submission with out-of-range coord...	Actual: [To be Tested] Status: [To be Tested]
TC-13: Frontend Form Reset Functionality Condition: User fills all 15 input fields and clicks "Reset Form"... Expected: All input inputs are cleared, probability meter hidden, and default status badge...	Actual: [To be Tested] Status: [To be Tested]
TC-14: End-to-End Latency Measurement Condition: Submit full transaction payload via browser client interface... Expected: Total round-trip time (HTTP POST + Preprocessing + Inference + JSON Response) < ...	Actual: [To be Tested] Status: [To be Tested]

REFERENCES

[1]

Python Software Foundation. Python 3.10+ Language Documentation. [Reference details to be verified]

Category: Programming Environment (Official documentation for standard libraries, datetime parsing, and mathematical calculations.)

[2]

Pallets Projects. Flask: A Python Microframework for Web Applications. [Reference details to be verified]

Category: Web Framework (Official technical documentation covering WSGI application routing, JSON request serialization, and template rendering.)

[3]

The pandas Development Team. pandas: Powerful Python Data Analysis Toolkit. [Reference details to be verified]

Category: Data Manipulation (Comprehensive guides on high-performance DataFrame operations, CSV ingestion, and datetime arithmetic.)

[4]

Harris, C.R. et al. Array Programming with NumPy. [Reference details to be verified]

Category: Numerical Mathematics (Documentation for vectorized array mathematics, trigonometric calculations, and Haversine distance computation.)

[5]

Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. [Reference details to be verified]

Category: Machine Learning Framework (Official reference documentation for HistGradientBoostingClassifier, compute_sample_weight, and train_test_split.)

[6]

Category Encoders Development Team. Category Encoders: A Collection of Scikit-Learn-Compatible Categorical Transformers. [Reference details to be verified]

Category: Feature Transformation (Documentation detailing the CatBoostEncoder implementation, target statistics calculations, and empirical prior probabilities.)

[7]

Joblib Development Team. Joblib: Running Python Functions as Pipeline Jobs. [Reference details to be verified]

Category: Object Serialization (Documentation covering optimized disk serialization and deserialization of NumPy and Scikit-learn estimators.)

[8]

Payment Card Industry Security Standards Council. PCI Data Security Standard (PCI-DSS) Specifications. [Reference details to be verified]

Category: Financial Security & Compliance (Industry security standards concerning the protection, masking, and non-storage of Primary Account Numbers.)

APPENDIX A: PROJECT REPOSITORY STRUCTURE

```
CreditCard-Fraud-Detection/
%
% % %   a p p . p y
% % %   r e q u i r e m e n t s . t x t
% % %   . g i t i g n o r e
%
% % %   m o d e l /
%       % % %   c a t b o o s t _ e n c o d e r . p k l
%       % % %   f i n a l _ f r a u d _ m o d e l . p k l
%       % % %   f r a u d _ t h r e s h o l d . p k l
%       % % %   h i s t _ g r a d i e n t _ b o o s t i n g . p k l
%       % % %   l o g i s t i c _ r e g r e s s i o n . p k l
%
% % %   n o t e b o o k s /
%       % % %   c l a s s _ b a l a n c e . p y
%       % % %   d a t a s e t _ i n s p e c t i o n . p y
%       % % %   e v a l u a t e _ f i n a l _ t h r e s h o l d . p y
%       % % %   p r e p r o c e s s i n g . p y
%       % % %   t e s t _ f i n a l _ p i p e l i n e . p y
%       % % %   t e s t _ f l a s k _ a p i . p y
%       % % %   t e s t _ r a w _ p i p e l i n e . p y
%       % % %   t h r e s h o l d _ a n a l y s i s . p y
%       % % %   t r a i n _ f i n a l _ p i p e l i n e . p y
%       % % %   t r a i n _ g r a d i e n t _ b o o s t i n g . p y
%       % % %   t r a i n _ l o g i s t i c _ r e g r e s s i o n . p y
%       % % %   v e r i f y _ p r o c e s s e d _ d a t a . p y
%
% % %   s t a t i c /
%       % % %   c s s /
%       %       % % %   s t y l e . c s s
%       %       % % %   j s /
%       %       % % %   a p p . j s
%
% % %   t e m p l a t e s /
%       % % %   i n d e x . h t m l
```